1. Dependency Injection (DI)

Dependency Injection is the design pattern where an object's dependencies are "injected" into it by an external entity (the IoC Container) rather than the object creating them itself.

- **The Problem:** In standard Java, if ServiceA needs ServiceB, you write ServiceB b = new ServiceB(). This hard-codes the dependency, making it impossible to swap ServiceB for a mock during testing.
- **The DI Solution:** You tell Spring, "I need a ServiceB," and Spring finds it in the container and hands it to you.
- **The Benefit:** Decoupling. Your code focuses on logic, not on managing the lifecycle of other objects.

---

2. Spring Bean Lifecycle

When Spring starts, it goes through a specific set of steps to get a Bean ready for use. Understanding this is vital for handling database connections or starting background tasks.

1. **Instantiation:** Spring creates the Java object (calling the constructor).
2. **Populate Properties:** Spring injects the dependencies (DI).
3. **Aware Interfaces:** If your bean implements BeanNameAware or ApplicationContextAware, Spring passes those IDs/Contexts to the bean.
4. **Pre-Initialization:** BeanPostProcessors take over.
5. **Initialization (@PostConstruct):** This is where you put your custom setup logic. It runs after the bean is fully constructed and injected.

+1

6. **Post-Initialization:** Further processing (like creating AOP Proxies for @Transactional).
7. **Ready:** The bean is now live in the container.
8. **Destruction (@PreDestroy):** Triggered when the ApplicationContext closes. Use this to close sockets or clear caches.

---

3. Constructor vs. Field Injection

While there are three types (Constructor, Setter, Field), the industry has shifted heavily toward **Constructor Injection**.

Field Injection (The "Old" Way)

Java

```
@Service
public class MyService {
  @Autowired
  private MyRepository repository; // Injected directly into the field
}
```

- **Cons:** It's hard to unit test because you can't easily inject a mock without using Reflection. It also allows for "Hidden Dependencies"—the class grows huge because adding a new @Autowired field is too easy.

Constructor Injection (The "Modern" Way)

Java

```
@Service
public class MyService {
  private final MyRepository repository;

  // Spring 4.3+ automatically autowires constructors if there's only one
  public MyService(MyRepository repository) {
    this.repository = repository;
  }
}
```

- **Pros:** 1. **Immutability:** You can mark dependencies as final.

2. **Testability:** You can simply call new MyService(mockRepo) in a JUnit test.

3. **Safety:** The compiler ensures the object cannot be created without its required dependencies (preventing NullPointerException).

4. **Circular Dependency Detection:** Spring will throw an error at startup if two beans try to inject each other via constructors, forcing you to fix a bad design.

---

4. Final Interview "Deep Dive" Tip

If asked about **Bean Scopes**, remember the two most common ones:

- **Singleton (Default):** One instance per Spring Container. Perfect for stateless services (like your Log Processors).
- **Prototype:** A new instance every time the bean is requested. Useful for stateful objects that aren't thread-safe.

One-Day Roadmap: Final Architecture Check

Question	Senior Answer Concept
Why use DI?	It facilitates the **Dependency Inversion Principle (DIP)** and makes code testable.
Why @PostConstruct?	Because the constructor hasn't seen the @Autowired fields yet; they are null until the Init phase.
Constructor vs Field?	Constructor is superior for immutability, safety, and unit testing.